

GUI & System Overview

This document explains **how the GUI is built**, what **functionality each user role** can perform, whether that functionality is *implemented*, how **data flows** through the layers, and the **purpose of every top-level folder**. It complements the deep-dive docs and JavaDoc site.

1 GUI Construction Philosophy

Layer	Purpose	Key Classes
Theming	Centralise colours, fonts, iconography so the rest of the UI is brand-agnostic	<code>ThemeManager</code> , <code>ThemeColors</code> , <code>ThemeFonts</code> , <code>ThemeIcons</code> , <code>ThemeSizes</code>
Reusable Widgets	Wrap Swing components to enforce look-and-feel & behaviour	<code>StyledButton</code> , <code>StyledTable</code> , <code>StyledTextField</code> , <code>ModernDialog</code> , <code>ActionButton</code> , <code>BasePanel</code>
Navigation Shell	Presents side/top bars & swaps screens via <code>CardLayout</code>	<code>MainFrame.navigateTo()</code> , dashboards & management panels
Domain Screens	High-level panels bound to services/models	<code>AdminDashboardPanel</code> , <code>MedicineManagementPanel</code> , <code>OrderManagementPanel</code> , etc.
Service Layer	Business logic & persistence orchestration	<code>PharmacyService</code> , <code>DoctorService</code> , etc.
Model Layer	POJOs representing domain entities	<code>User</code> , <code>Order</code> , <code>Medicine</code> , ...

Design patterns employed: **Singleton** (`ThemeManager`), **Factory** (`ThemeIcons`), **Observer** (`PropertyChangeSupport` in models), **MVC/MVP hybrid** (screen panels delegate to services), **Template Method** (`BasePanel`, `BaseDashboard`).

2 User Roles & Functional Matrix

Role	GUI Feature	Screen / Panel	Implemented	Notes
Admin	Dashboard KPIs	<code>AdminDashboardPanel</code>		Cards, charts, recent activity
	User Management (CRUD)	<code>UserManagementPanel</code>		Search, filter, add, edit, delete

Role	GUI Feature	Screen / Panel	Implemented	Notes
	Medicine Catalogue	MedicineManagementPanel	✓	CSV import/export, stock alert badges
	Orders Overview	OrderManagementPanel	✓	Status filter, view/process
	Reports / Analytics	ReportsPanel	✓	Generates CSV/PDF, charts
Doctor	Dashboard	DoctorDashboard	✓	Patient stats
	Patient List	PatientListPanel	✓	Table with search
	Prescriptions	PrescriptionsPanel	✓	Create & sign prescriptions
	Consultations	ConsultationsPanel	✓	Appointment list (basic)
	Medical Records	MedicalRecordsPanel	⚠ Partial	View-only, edit planned
Pharmacist	Dashboard	PharmacistDashboardPanel	✓	Incoming orders & Rx KPIs
	Medicine Inventory	MedicineManagementPanel (shared)	✓	Restricted actions
	Order Queue	OrderManagementPanel	✓	Process / validate
	Prescription Validation	PrescriptionManagementPanel	✓	Approve / reject
Patient	Dashboard	PatientDashboardPanel	✓	Activity summary
	Browse Medicines	MedicineManagementPanel (read-only)	✓	Add to cart planned
	View Orders	OrderManagementPanel (filtered)	⚠ Partial	Status tracking only
	Prescriptions	(future)	■ Planned	Will display doctor prescriptions

3 Data Processing & Workflow

```
sequenceDiagram
    participant UI as ScreenPanel
    participant Service as PharmacyService
    participant Model as Domain Models
    participant IO as File-Storage (JSON/CSV)
    UI->>Service: getOrders()
    Service->>IO: load order.json
    IO-->>Service: List<Order>
    Service-->>UI: filtered orders
    UI->>UI: render StyledTable
    UI->>Service: updateStatus(orderId)
    Service->>Model: mutate Order
    Service->>IO: save order.json
```

1. **UI Layer (Swing Panel)** requests data via a service façade.
2. **Service Layer** loads domain objects from JSON (or caches) and enforces business rules.
3. **Model Layer** changes state; **PropertyChangeSupport** notifies listeners if bound to UI.
4. Service persists the change back to flat-file storage (JSON/CSV). A database adapter can be swapped in later.

4 Folder Rationale

Folder	Why it exists / What goes inside
<code>src/gui/theme</code>	Consolidates theme artefacts so we <i>don't</i> sprinkle colour/size magic-numbers across widgets. Enables brand re-skin by editing a handful of classes.
<code>src/gui/components</code>	Home for all reusable widgets. No screen-specific logic allowed here. Guarantees consistent styling (e.g., StyledButton) across dashboards.
<code>src/gui/screens</code>	Generic screens (login, registration) that don't belong to a specific role.
<code>src/gui/admin,</code> <code>doctor, pharmacist,</code> <code>dashboard</code>	Role-scoped or domain-scoped panels, keeping features discoverable. Each extends BasePanel so spacing & colours are uniform.
<code>src/gui/navigation</code>	(Obsolete) previously held SidebarPanel ; we now route via top-bar buttons.
<code>src/models</code>	Pure data objects—no Swing or persistence code. Serializable to JSON/XML.

Folder	Why it exists / What goes inside
<code>src/services</code>	Business logic + DAO responsibilities. Shields the GUI from file format / DB changes.
<code>src/data</code>	Seed JSON / CSV samples for demo purposes.
<code>icons</code>	All PNG/SVG assets addressed by <code>ThemeIcons</code> .
<code>docs</code>	Markdown & diagrams—kept versioned with code.

5 Consistency Across Dashboards & Screens

- **Inheritance** – All dashboards extend `BaseDashboardPanel`, which defines:
 - unified header band (title + icon)
 - reusable `DashboardStat` & `DashboardActivity` inner classes for KPI cards & activity feeds
 - common padding & surface colour.
- **Theming** – Buttons, tables, dialogs consult `ThemeColors` and `ThemeFonts`, so a palette swap propagates everywhere.
- **Layout Primitives** – `LayoutUtils`, `Flex` helpers abstract `Box` / `GridBag` boilerplate, ensuring spacing harmony.
- **Navigation** – `MainFrame.navigateTo(route)` is the single entry point; no panel constructs another panel directly, which prevents divergent flows.

6 How to Verify Implementation Coverage

Run the application and log in with each demo user account (provided in `data/users.json`). Use the matrix in Section 2 to tick off implemented features; partial screens will show a banner "Coming Soon" message coded in their `initializeComponents()`.

Automated test stubs exist in `test/integration/` to crawl the GUI (via AssertJ-Swing) and assert that every route renders without exception—helpful CI safety-net.

Last updated: {{DATE_AUTOGEN}}